

# Teaching Statement

Md Abdullah Al Mazid

Florida International University

## Teaching Philosophy

---

My goal as an instructor is to help students move from knowing terminology to being able to reason about a system, explain why it behaves as it does, and use that understanding to solve unfamiliar problems. Computer science courses can easily become collections of abstractions: normalization rules, transaction properties, process scheduling, memory hierarchies, neural architectures, or optimization objectives. I therefore organize my teaching around three habits of mind: **build conceptual models**, **connect those models to concrete systems**, and **practice explaining and applying them**.

I have seen that students often struggle not because they are incapable of learning a topic, but because a missing conceptual link makes later material appear arbitrary. My first responsibility is therefore to identify that missing link. I use diagrams, progressively more complex examples, small counterexamples, and comparisons between alternative designs to make the structure of a topic visible. Once students understand the structure, I ask them to apply it through discussion, problem solving, implementation, or analysis of real systems. My aim is for students to leave a course with transferable reasoning skills rather than a collection of memorized answers.

## Teaching Experience at FIU

---

At Florida International University, I have served as a Graduate Teaching Assistant for **CIS 5027 - Computer Systems Fundamentals** and **COP 5725 - Principles of Database Management Systems** in Fall 2024, Fall 2025, and Fall 2026. My responsibilities have included instructional and lecture support, office hours, student discussions, course assessment, and instructor-directed course design and review. I have also reviewed instructional materials, assignments, and assessments at the professor's request.

These experiences have shown me the importance of offering students more than one route to understanding. During office hours and individual or group discussions, I first ask students to describe how they currently understand the problem. That makes it easier to distinguish a conceptual misunderstanding from a small procedural error. For database topics, for example, a student who is struggling with a query may actually be confused about the relational structure or the meaning of a join. For systems topics, an incorrect answer may come from mixing up layers of abstraction. I try to reconstruct the idea with the student rather than simply provide the final solution.

I also value discussion during and around lectures. When students ask questions, I use those questions to expose assumptions that may be hidden in a formal definition or slide. In a graduate classroom, this is particularly useful because students arrive with varied backgrounds in programming, systems, databases, and mathematics. A short clarification that links new material to something familiar can make the rest of a lecture substantially more accessible.

## Connecting Theory and Practice

---

Before beginning my PhD, I worked for more than three years in professional software development, including as a Senior Software Engineer at Enosis Solutions. That experience strongly influences how I teach. I want students to understand why a theoretical concept matters in an actual system: why database design affects maintainability, why inefficient queries become operational problems, why interfaces and abstractions matter in large codebases, and why performance, correctness, and reliability must often be considered together.

I do not treat industry examples as a replacement for theory. Instead, I use them to motivate theory and to show students where a concept becomes consequential. Similarly, my current research in machine

learning, remote sensing, multimodal learning, and scientific computing provides examples of how core computer-science ideas recur in research practice. Data structures, algorithmic reasoning, numerical computation, data modeling, system design, experimentation, reproducibility, and careful evaluation remain essential even when the application involves modern AI.

## Competitive Programming and Algorithmic Problem Solving

---

Competitive programming played an important role in my own development as a computer scientist. As an undergraduate, I participated in collegiate programming competitions including the **ICPC Dhaka Regional Final** and the **National Collegiate Programming Contest**, along with other inter-university contests, and spent substantial time practicing algorithmic problems on online programming platforms. The most lasting value of that experience was not the contests themselves, but the habits they developed: decomposing an unfamiliar problem, identifying the appropriate abstraction, reasoning about time and space complexity, considering edge cases, translating an idea into correct code, and debugging systematically when the first approach fails.

Those habits later proved valuable in professional software engineering. Real software problems are generally larger and less neatly defined than programming-contest problems, but the ability to break a complex requirement into manageable pieces, reason carefully about efficiency and correctness, and test assumptions is directly transferable. The same mindset continues to help me in research, where new methods frequently require translating mathematical ideas into implementations, diagnosing unexpected behavior, designing efficient experiments, and determining whether a failure comes from the underlying idea, the data, or the implementation.

I therefore see competitive programming as a valuable **co-curricular pathway for developing problem-solving ability**, particularly for students who want additional practice beyond regular coursework. As a faculty member, I would encourage interested students to participate in programming contests and would support the development of an active problem-solving community through regular practice sessions, student-led problem discussions, mock contests, and preparation for collegiate competitions. I would structure practice so that students can progress from approachable problems involving basic programming and data structures to more advanced topics such as dynamic programming, graphs, greedy algorithms, and algorithmic optimization.

Equally important, I would emphasize discussion after solving a problem. Students should be able to explain not only *what* solution worked, but *why* it worked, what alternative approaches were possible, where an inefficient solution failed, and how the implementation could be improved. I would also make clear that competitive programming is one avenue for developing computational problem-solving skills rather than a measure of who does or does not belong in computer science. Students should be able to enter such activities at different levels of experience and use them as opportunities to practice, collaborate, and improve.

## Assessment, Feedback, and Responsible Use of AI

---

I view assessment as part of instruction rather than merely a mechanism for assigning grades. Good assignments should reveal whether students can transfer a concept to a new setting. I prefer a progression from foundational questions to tasks that require explanation, design choices, debugging, or comparison of alternatives. When reviewing course materials and assessments, I pay particular attention to whether the question is testing the intended learning objective and whether the expected answer follows clearly from the material students were asked to learn.

Feedback is most effective when students can act on it. In office hours and discussions, I therefore focus on the reasoning step that needs correction. When possible, I ask students to try the next step themselves after we identify the issue. This turns feedback into practice and helps students become more

independent learners.

Generative AI creates both opportunities and challenges for computing education. I do not think students benefit from pretending these tools do not exist. Instead, I would teach them to use AI critically: verify generated code and explanations, identify unsupported assumptions, test outputs, document tool use when required, and understand that producing an answer is not the same as understanding it. Assessments should increasingly ask students to explain design decisions, analyze failures, defend tradeoffs, and demonstrate understanding in ways that cannot be replaced by unexamined tool output.

## **Inclusive and Supportive Learning**

---

Students participate more effectively when they can ask questions without being made to feel that uncertainty is a failure. I try to create an environment in which questions are treated as part of technical work. During office hours and group discussions, I adapt explanations to the student's starting point and avoid assuming that everyone has encountered the same tools or examples before. Clear expectations, transparent grading criteria, and multiple opportunities to ask questions help reduce unnecessary barriers while maintaining the same academic standards.

My own academic path has taken me through different educational and professional environments, from computer science training in Bangladesh to graduate research and teaching in the United States. This has made me attentive to differences in prior preparation, communication style, and familiarity with classroom conventions. I want students to feel that they belong in technical discussions while also being challenged to develop rigorous reasoning and professional independence.

## **Courses I Can Contribute To**

---

I am prepared to contribute to undergraduate and graduate teaching in **programming, data structures and algorithms, database systems, computer systems, software engineering, machine learning, deep learning, computer vision, and data science**. My research also positions me to develop advanced courses or modules in **multimodal machine learning, GeoAI, remote sensing, scientific machine learning, and interpretable AI**. I would also be interested in supporting programming-contest preparation and other co-curricular activities that give students additional opportunities to strengthen their algorithmic problem-solving skills.

Across these areas, my objective would remain the same: give students a strong conceptual foundation, connect it to real computational problems, and help them become confident, independent problem solvers.